



Applied Electronic project
Dr Pasharavesh

**Smart Plant Irrigation System with Remote Flow
Control**

Aryan Molazadeh 401107527
Mohammad Sadegh Dehghani 401107013
Peyman Shahheidari 401107108

Contents

1- Project Statement.....	2
2- Review of Components and Tools Used	3
2 – 1 - Arduino UNO	3
2-2- DC Water pump	5
2-3- L298N Motor Driver Module	6
2-4- Soil Moisture Sensor	7
2-5-Water Level Sensor	9
2-6- ACS712 Current Sensor.....	10
2-7-Buzzer.....	11
2-8- WiFi Module (ESP8266).....	12
2-9- Mobile Application RemoteXY	13
3-System Operation Description.....	14
3-1- Component Connection Diagram.....	14
3-2- Final Project Code:.....	16
4-Conclusion	19

1- Project Statement

Project Definition:

In this project, an automatic plant irrigation system is designed using an Arduino board. This system allows the user to, at any time, view the soil moisture level and water flow rate on their mobile phone screen and adjust the flow rate if necessary.

Irrigation is performed automatically and will continue until the soil moisture reaches the desired level set by the user.

Project Description:

In this project, a water pump (DC motor) is driven using an L298 module. The soil moisture level is measured in real-time by a soil moisture sensor, and the sensor's output signal is fed to the Arduino board. Based on the moisture level and the set desired water flow rate, the water pump is activated and will continue operating until the required moisture level is achieved.

During irrigation, the water flow rate must also be measured and displayed to the user. To measure the flow rate, there is no need for a dedicated flow sensor; instead, an appropriate method using a current sensor, water level sensor, or other available sensors should be selected and implemented for measuring and calculating the water flow rate.

For user-system communication, enabling the user to view moisture and flow rate values, as well as adjust the desired flow rate and soil moisture levels, a WiFi module will be used. The user can connect their phone to the system's WiFi to receive necessary information and send commands to the Arduino board, thereby changing the desired water flow rate and moisture level, and monitoring the irrigation status.

Furthermore, if the water reservoir level drops or there is an unauthorized increase in the pump's current (e.g., due to a blockage in the water path), the system will automatically shut off the pump and alert the user via a buzzer.

2- Review of Components and Tools Used

2 – 1 - Arduino UNO

The Arduino UNO is a microcontroller board based on the ATmega328P. This board is capable of reading input from various sensors and controlling outputs such as motors, pumps, or displays. In this project, the Arduino acts as the central control unit. All sensors (soil moisture, water level, current) are connected to its analog pins, and their values are read by the Arduino. Additionally, the water pump is controlled via digital and PWM pins. Furthermore, the Arduino manages the communication between the WiFi module and the RemoteXY application.



➤ Related Code:

```
void setup() {  
  RemoteXY_Init ();  
  Serial.begin(115200);  
  pinMode(4,OUTPUT);  
  pinMode(12,OUTPUT);  
  pinMode(13,OUTPUT);  
  RemoteXY.slider_01= 100;  
  RemoteXY.slider_02= 50;  
  RemoteXY.strings_01=0;  
}
```

This section of the code executes at the beginning of the program. Here, the RemoteXY and serial communication are initialized, output pins for motor control and alerts are defined, and initial values for sliders and user interface variables are set.

RemoteXY_Init (): This command initializes the RemoteXY module defined on the Arduino board, enabling it to establish a WiFi connection with a mobile phone.

`Serial.begin(115200)`: This command initializes serial communication at a baud rate of 115200 bits per second. This serial connection is used both for debugging (displaying data on the serial monitor) and for certain functions of RemoteXY.

Once we have verified the code's correctness, finalized it, and no longer require a connection to the computer, we comment out this line by adding `//` at the beginning of the line. This helps reduce the potential for errors in the WiFi module.

`pinMode(4, OUTPUT)`: Pin number 4 is configured as an output. In the code, this pin is used to activate the alert buzzer (or to turn off the pump in error conditions).

`pinMode(12, OUTPUT)` and `pinMode(13, OUTPUT)`: These two pins are used to control the rotation direction of the pump motor via the L298N driver. In normal operation, one is set to HIGH and the other to LOW to rotate the motor in the desired direction.

`RemoteXY.slider_1 = 100` : The initial value of the first slider in the RemoteXY application is set to 100 (this slider determines the pump flow rate).

`RemoteXY.slider_02= 50` : The initial value of the second slider is set to 50 (this slider determines the soil moisture threshold).

`RemoteXY.strings_01=0` : The initial value of this variable is set to zero. This variable is used to display the system status in the application (e.g., Normal, Low Water Level, or Excessive Flow).

2-2- DC Water pump

A DC water pump is an electromechanical device that operates on direct current (DC) voltage. By rotating an internal impeller, it draws water from the inlet and directs it toward the soil through the outlet. The pump's speed is directly dependent on the voltage and power applied to the motor.

In this system, the voltage and speed of the pump—and consequently, the water flow rate—can be adjusted using a PWM (Pulse Width Modulation) signal sent from the Arduino to the motor driver. The pump's operating intensity is controlled either via the RemoteXY application or based on soil conditions (dry or moist).



➤ Related code:

```
analogWrite(10,moisturetopump);
```

The `analogWrite` command generates a PWM signal on pin number 10.

The `(moistureToPump)` variable determines the PWM signal value and ranges from 0 to 255. A higher value of `(moistureToPump)` increases the effective voltage applied to the motor, resulting in higher pump rotation speed and consequently a greater water flow rate.

If `(moistureToPump)` is set to zero, the PWM signal is effectively cut off and the pump will turn completely off.

2-3- L298N Motor Driver Module

The L298N module is an H-Bridge-based motor driver used to control DC and stepper motors. This module can control both the rotation direction and speed of the motor (via PWM signals). By using this driver, the motor can be supplied with sufficient voltage and current without overloading the Arduino.

In this project, the L298N acts as an interface between the Arduino and the DC water pump. The Arduino sends HIGH/LOW and PWM control signals to the L298N, and the driver is responsible for operating the pump motor according to these signals. Arduino pins 12 and 13 determine the motor direction, while pin 10 sets the PWM intensity (pump speed).



➤ related code:

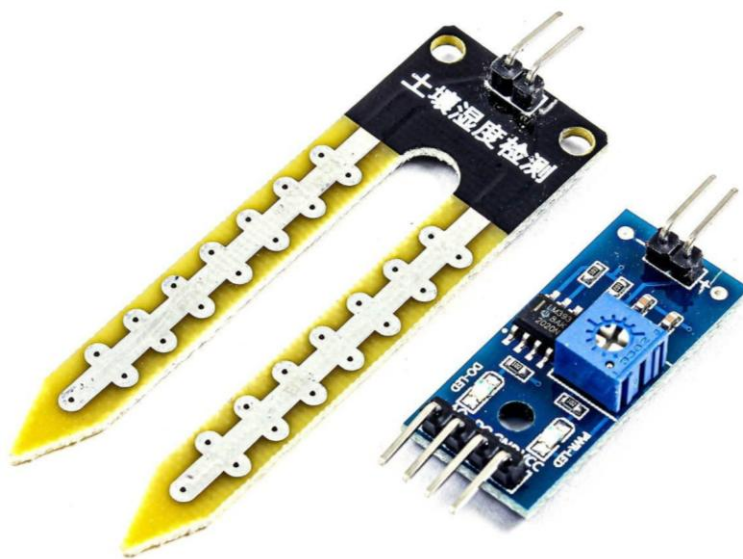
```
digitalWrite(4, LOW);  
digitalWrite(13, LOW);  
digitalWrite(12, HIGH);  
  
analogWrite(10, moistureTopump);
```

- ✓ The L298N both maintains the pump's fixed direction and enables motor speed control via PWM. In this setup, the Arduino's output pins determine the motor's operating state.
- ✓ `digitalWrite(13, LOW)` and `digitalWrite(12, HIGH)` : This configuration causes the motor to rotate only in one specific direction (the direction set for water pumping).
- ✓ `digitalWrite(4, LOW)` : Pin number 4 is kept LOW here, but under error conditions (e.g., low water level or excessive current), it is set to HIGH to activate the buzzer and simultaneously shut off the pump.
- ✓ Alongside these instructions, as explained in the pump section, the motor rotation speed is controlled by the `digitalWrite(10, moistureToPump)` command.

2-4- Soil Moisture Sensor

The soil moisture sensor consists of two metal probes that are inserted into the soil. These probes act as the two ends of a resistor. When the soil is dry, the resistance increases, and the sensor's output voltage decreases. Conversely, when the soil is moist, the resistance decreases, and the output voltage rises. The sensor provides an analog output to the Arduino.

In this project, the soil moisture sensor reports the actual moisture level to the Arduino. The Arduino then compares the read value with a threshold set by the user via the application and decides whether to turn the water pump on or off.



➤ Related code:

```
int moisturevalueR=analogRead(A0);
moisturevalue= 1023-moisturevalueR;
currentvalue=analogRead(A4);
if (moisturevalue < RemoteXY.slider_02*10.23){
    if(RemoteXY.slider_02==50){
        moisturetopump = map(moisturevalueR, 0, 1023, 0, 255);
    } else {
        moisturetopump=map(RemoteXY.slider_01,0,100,0,255);
    }
} else{
    moisturetopump=0;
}
```

- ✓ **analogRead(A0):** Reads the analog voltage value from the sensor (range 0 to 1023).
- ✓ **moistureValue = 1023 - moistureValueR:** The value is inverted so that a higher number indicates greater moisture.

- ✓ `if (moistureValue < RemoteXY.slider_02 * 10.23)`: Compares the read value with the threshold set by the user in the app (`RemoteXY.slider_02` ranges from 0 to 100). The factor 10.23 is used to convert the slider value (100) to the ADC scale (1023), since $1023 / 100 = 10.23$.
- ✓ `if (RemoteXY.slider_02 == 50)`: Checks if the threshold slider is exactly 50. If true, the system enters automatic mode (adjusting flow rate based on sensor readings); otherwise, manual mode is assumed, and the flow rate is read from `slider_01`.
- ✓ `moistureToPump = map(moistureValueR, 0, 1023, 0, 255)`: In "Auto" mode, the PWM value for the pump is automatically calculated from the raw sensor reading. The `moistureValueR` is mapped from 0–1023 to 0–255 for use with `analogWrite`.
- ✓ `moistureToPump = map(RemoteXY.slider_01, 0, 100, 0, 255)`: In manual mode (when `slider_02` is not 50), the pump flow rate is set via the first app slider (0–100) and mapped to the PWM range of 0–255.
- ✓ `else { moistureToPump = 0; }`: If the threshold condition is not met (i.e., current moisture is above or equal to the threshold), `moistureToPump` is set to 0, turning the pump off.

2-5-Water Level Sensor

The water level sensor is a module with multiple metal electrodes that detects the presence or absence of water based on its conductivity. When water reaches the sensor level, the electrodes connect and generate an analog signal. The lower the water level, the lower the output voltage.

In this project, the water level sensor serves to protect the pump. If the water level in the tank drops or there is insufficient water, the Arduino detects this via the sensor and immediately shuts off the pump. It also sends the necessary alert to the user through the RemoteXY application.



➤ related code:

```
waterNvalue=analogRead(A1);
if (waterNvalue<100){
  RemoteXY.strings_01=1;
  moisturetopump=0;
  digitalWrite(4,HIGH);
}
waterPvalue=waterNvalue;
RemoteXY_delay(1000);
waterNvalue=analogRead(A1);
moisturevalue=map(moisturevalue,0,1023,0,100);
dtostrf((waterPvalue-waterNvalue)*0.52,0,1,RemoteXY.value_01);
```

- ✓ `waterNvalue = analogRead(A1)`: The water level sensor signal is read from pin A1. This value ranges from 0 to 1023.
- ✓ `if (waterNvalue < 100)`: If the value is less than 100, it means the water level is very low (nearly empty tank). In this case:

- ✓ `RemoteXY.strings_01 = 1`: An alert message is set for the application.
- ✓ `moistureToPump = 0`: The pump stops to prevent damage from running dry.
- ✓ `digitalWrite(4, HIGH)`: Pin 4 is activated, triggering the buzzer to sound an alarm.
- ✓ `waterPvalue = waterNvalue`: The current sensor value is stored as the previous value.
- ✓ `RemoteXY_delay(1000)`: A one-second delay to calculate changes in the water level inside the tank.
- ✓ `waterNvalue = analogRead(A1)`: The new water level sensor value is read.
- ✓ `dtostrf((waterPvalue - waterNvalue) * 0.52, 0, 1, RemoteXY.value_01)`: The difference between the previous and new sensor values is calculated and multiplied by 0.52 (this coefficient is derived from multiplying the tank's cross-sectional area by the water level sensor calibration factor). This value is displayed to the user and represents an estimate of the water flow rate (i.e., the speed of water outflow from the tank).

2-6- ACS712 Current Sensor

In this project, the ACS712 measures the water pump's current consumption. If the current exceeds the permissible limit (for example, due to pump impeller blockage or failure in the water path), the Arduino detects this abnormal condition, immediately shuts off the pump, and sends the necessary alert to the user.



➤ related code:

```
currentvalue=analogRead(A4);
if (currentvalue>700){
    RemoteXY.strings_01=2;
    moisturetopump=0;
    digitalWrite(4,HIGH);
}
```

- `analogRead(A4)`: The output value of the current sensor is read from pin A4.
- `if (currentValue > 700)`: If this value exceeds 700, it indicates that unauthorized current has been detected. In this case:

- RemoteXY.strings_01 = 2: The application receives an error message with the code "High Current".
- moistureToPump = 0: The pump is turned off.
- digitalWrite(4, HIGH): The buzzer is activated.

2-7-Buzzer

In this project, the buzzer serves as an alert system. It activates whenever any of the following conditions occur:

- The water tank level drops too low
- The pump's current consumption exceeds the permitted limit

This way, the user can be notified of system issues even without checking the application.



➤ **related code:**

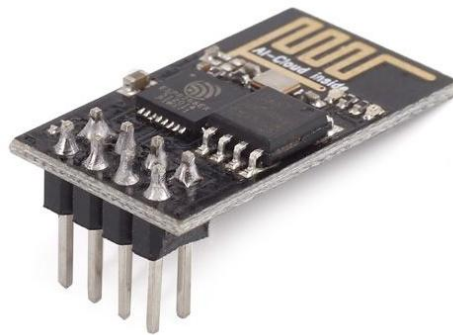
```
digitalWrite(4,HIGH)
```

When pin 4 is set to HIGH, the buzzer activates and produces an alert sound. This state only occurs under abnormal conditions (such as low water level or excessive current). Under normal conditions, pin 4 remains LOW and the buzzer is off.

2-8- WiFi Module (ESP8266)

In this project, the ESP8266 is responsible for connecting the Arduino to the RemoteXY application. This enables the user to, via their mobile phone:

- View the soil moisture level and water flow rate
- Adjust the soil moisture threshold
- Manually change the pump flow rate
- Receive alert statuses (such as low water level or excessive current)



➤ related code :

```
#define REMOTEXY_MODE__ESP8266_HARDSERIAL_POINT

#define REMOTEXY_SERIAL Serial
#define REMOTEXY_SERIAL_SPEED 115200
#define REMOTEXY_WIFI_SSID "RemoteXY"
#define REMOTEXY_WIFI_PASSWORD "12345678"
#define REMOTEXY_SERVER_PORT 6377

#include <RemoteXY.h>
```

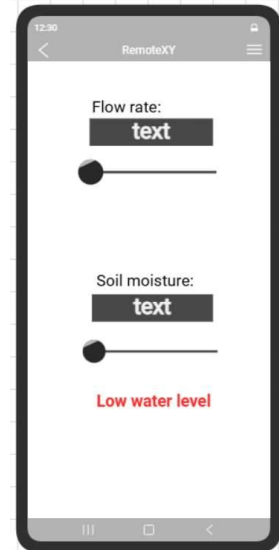
- `#define REMOTEXY_MODE__ESP8266_HARDSERIAL_POINT`: Specifies that the communication between the Arduino and the application will be established via the ESP8266 module using serial connection.
- `#define REMOTEXY_SERIAL Serial` and `#define REMOTEXY_SERIAL_SPEED 115200`: Define the serial port and data transmission speed at 115200 bits per second.
- `#define REMOTEXY_WIFI_SSID "RemoteXY"` and `#define REMOTEXY_WIFI_PASSWORD "12345678"`: The WiFi network created by the ESP8266 will be named "RemoteXY" with the password "12345678". The user's phone must connect to this network.
- `#define REMOTEXY_SERVER_PORT 6377`: The port used for communication between the application and the Arduino.
- `#include <RemoteXY.h>`: The RemoteXY library for managing all communications and the application's user interface.

2-9- Mobile Application RemoteXY

In this project, the RemoteXY application serves as the user interface. The user can:

- View the soil moisture level as a percentage
- Read the flow rate value
- Adjust the soil moisture threshold using a slider
- Manually control the pump flow rate
- Receive alerts (low water level or excessive current)

➤ **related code:**



```
#pragma pack(push, 1)
uint8_t RemoteXY_CONF[] =
{ 255,2,0,23,0,128,0,19,0,0,0,0,31,1,106,200,1,1,7,0,
  67,27,25,54,12,69,16,26,11,4,22,43,66,11,128,8,26,129,28,17,
  31,7,0,24,70,108,111,119,32,114,97,116,101,58,0,129,30,93,43,7,
  0,24,83,111,105,108,32,109,111,105,115,116,117,114,101,58,0,67,28,102,
  53,12,69,16,26,11,74,27,144,56,9,21,2,31,37,64,76,111,119,32,
  119,97,116,101,114,32,108,101,118,101,108,0,37,64,72,105,103,104,32,99,
  117,114,114,101,110,116,0,4,24,122,64,10,128,8,26 };

struct {

  int8_t slider_01;
  int8_t slider_02;
  char value_01[11];
  char value_02[11];
  uint8_t strings_01;
  uint8_t connect_flag;
```

The RemoteXY_CONF[] array is the graphical interface descriptor generated by the RemoteXY website. The RemoteXY structure contains all variables exchanged between the application and the Arduino:

- slider_01: First slider for setting the pump flow rate (0 to 100).
- slider_02: Second slider for setting the soil moisture threshold (0 to 100).
- value_01: String value for displaying the calculated water flow rate.
- value_02: String value for displaying the soil moisture percentage.
- strings_01: System status variable (0 = Normal, 1 = Low Water Level, 2 = Unauthorized Current).
- connect_flag: Connection status (1 = Connected, 0 = Disconnected).

3-System Operation Description

3-1- Component Connection Diagram

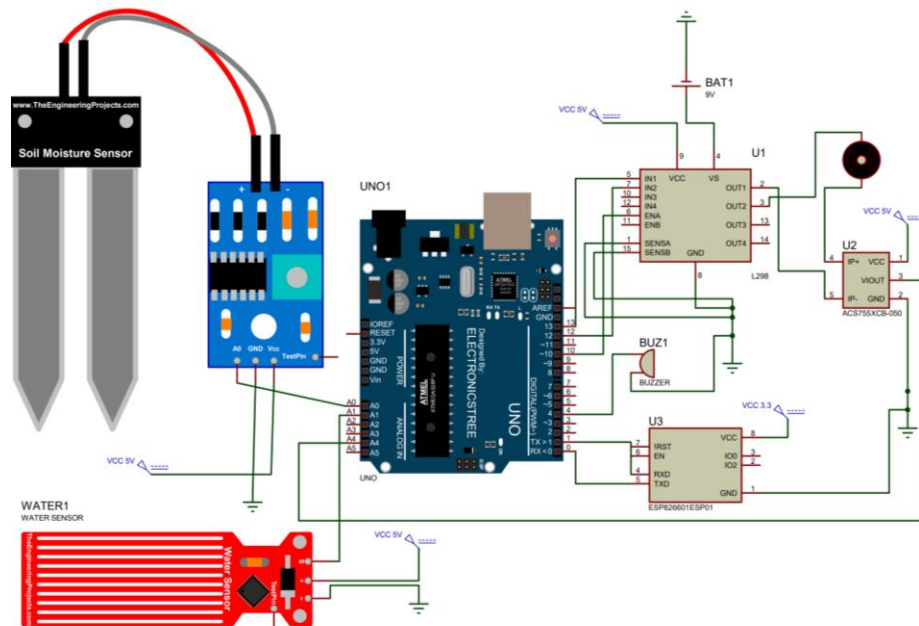
- **Arduino UNO:** All components are connected to the digital or analog pins of the Arduino.
- **Water Pump and L298N Driver Module:** The water pump is connected to the output of the L298N module. The input pins of the L298N for motor control are connected to the Arduino as follows:

- Pin 12 of Arduino → IN1 pin of L298N module
- Pin 13 of Arduino → IN2 pin of L298N module
- Pin 10 of Arduino → ENA pin of L298N module

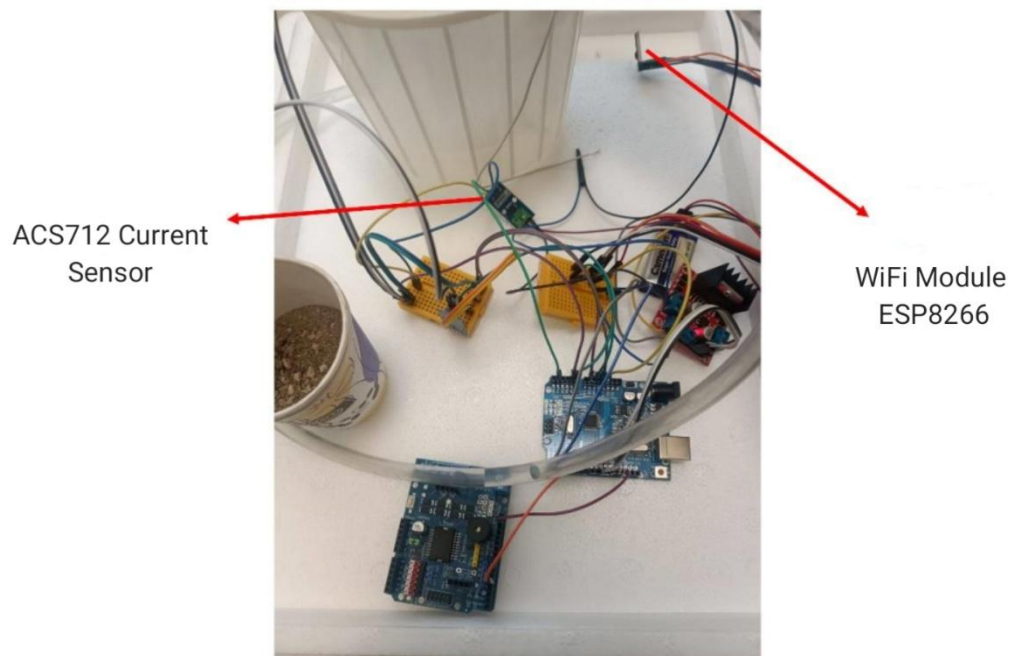
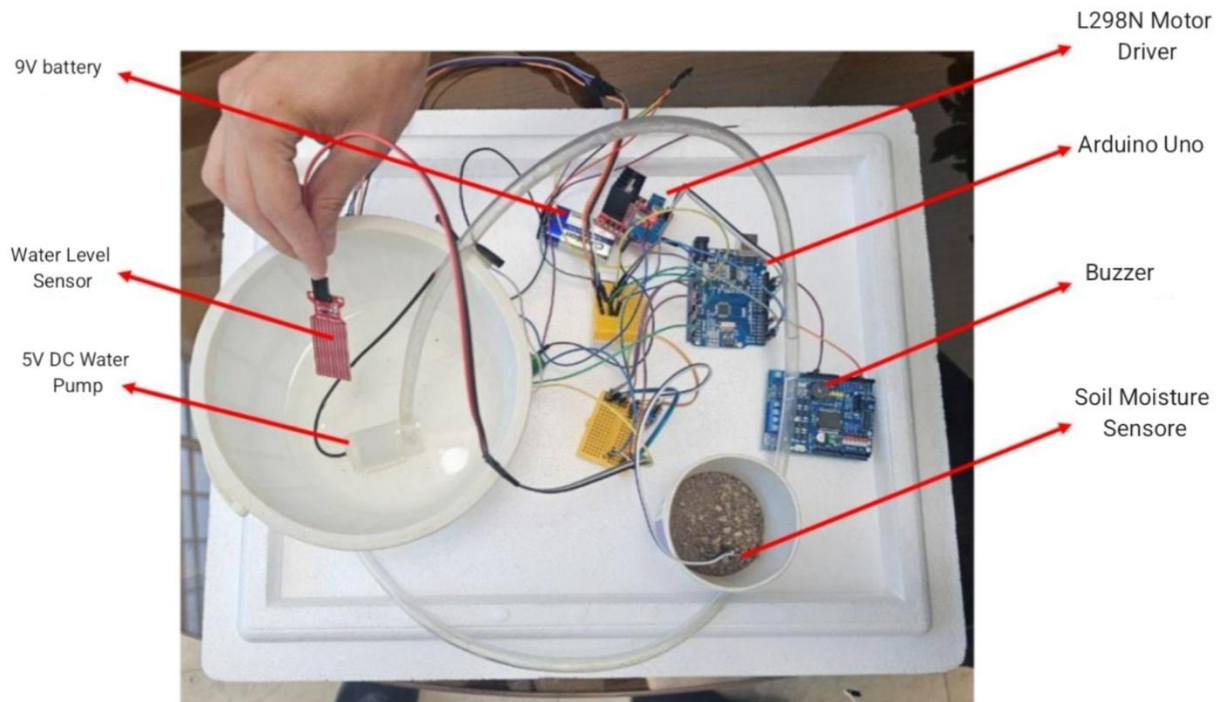
Pump Power Supply: Provided by a 9V DC source connected to the L298N module.

- **Soil Moisture Sensor:** Its analog output is connected to Arduino pin A0. The sensor is powered by the Arduino's 5V and GND.
- **Current Sensor:** Its analog output is connected to Arduino pin A4.
- **Water Level Sensor:** Its analog output is connected to Arduino pin A1.
- **WiFi Module:** Connected to the Arduino via serial pins RX and TX.
- **Buzzer:** The buzzer pin is connected to Arduino pin 4.

System Circuit Schematic:



Final Project Setup:



3-2- Final Project Code:

```
int moisturevalue ;

int moisturetopump ;
int currentvalue;
int waterPvalue;
int waterNvalue;

// you can enable debug logging to Serial at 115200
// #define REMOTEXY__DEBUGLOG

// RemoteXY select connection mode and include library
#define REMOTEXY_MODE__ESP8266_HARDSERIAL_POINT

// RemoteXY connection settings
#define REMOTEXY_SERIAL Serial
#define REMOTEXY_SERIAL_SPEED 115200
#define REMOTEXY_WIFI_SSID "RemoteXY"
#define REMOTEXY_WIFI_PASSWORD "12345678"
#define REMOTEXY_SERVER_PORT 6377

#include <RemoteXY.h>

// RemoteXY GUI configuration
#pragma pack(push, 1)
uint8_t RemoteXY_CONF[] = // 135 bytes
{ 255,2,0,23,0,128,0,19,0,0,0,0,31,1,106,200,1,1,7,0,
  67,27,25,54,12,69,16,26,11,4,22,43,66,11,128,8,26,129,28,17,
  31,7,0,24,70,108,111,119,32,114,97,116,101,58,0,129,30,93,43,7,
  0,24,83,111,105,108,32,109,111,105,115,116,117,114,101,58,0,67,28,102,
  53,12,69,16,26,11,74,27,144,56,9,21,2,31,37,64,76,111,119,32,
  119,97,116,101,114,32,108,101,118,101,108,0,37,64,72,105,103,104,32,99,
  117,114,114,101,110,116,0,4,24,122,64,10,128,8,26 };

// this structure defines all the variables and events of your control interface
struct {

    // input variables
    int8_t slider_01; // from 0 to 100
    int8_t slider_02; // from 0 to 100

    // output variables
    char value_01[11]; // string UTF8 end zero
    char value_02[11]; // string UTF8 end zero
    uint8_t strings_01; // from 0 to 2
```

```

    // other variable
    uint8_t connect_flag; // =1 if wire connected, else =0

} RemoteXY;
#pragma pack(pop)

////////////////////////////////////
//          END RemoteXY include          //
////////////////////////////////////

void setup() {
    RemoteXY_Init ();
    Serial.begin(115200);
    pinMode(4,OUTPUT);
    pinMode(12,OUTPUT);
    pinMode(13,OUTPUT);
    RemoteXY.slider_01= 100;
    RemoteXY.slider_02= 50;
    RemoteXY.strings_01=0;
}

void loop() {
    RemoteXY_Handler ();
    digitalWrite(4,LOW);
    digitalWrite(13,LOW);
    digitalWrite(12,HIGH);
    int moisturevalueR=analogRead(A0);
    moisturevalue= 1023-moisturevalueR;
    currentvalue=analogRead(A4);
    if (moisturevalue < RemoteXY.slider_02*10.23){
        if(RemoteXY.slider_02==50){
            moisturetopump = map(moisturevalueR, 0, 1023, 0, 255);
        } else {
            moisturetopump=map(RemoteXY.slider_01,0,100,0,255);
        }
    } else{
        moisturetopump=0;
    }
    if (currentvalue>700){
        RemoteXY.strings_01=2;
        moisturetopump=0;
        digitalWrite(4,HIGH);
    }
    waterNvalue=analogRead(A1);
    if (waterNvalue<100){

```

```
    RemoteXY.strings_01=1;
    moisturetopump=0;
    digitalWrite(4,HIGH);
}
waterPvalue=waterNvalue;
RemoteXY_delay(1000);
waterNvalue=analogRead(A1);
moisturevalue=map(moisturevalue,0,1023,0,100);
dtostrf(moisturevalue, 0, 1, RemoteXY.value_02);
dtostrf((waterPvalue-waterNvalue)*0.52,0,1,RemoteXY.value_01);
analogWrite(10,moisturetopump);
}
```

4-Conclusion

In this project, a smart plant irrigation system was designed and implemented. Using Arduino and a set of sensors and modules, the system successfully automated the irrigation process while remaining under user control. The results obtained from testing demonstrated that:

- The system is capable of measuring soil moisture and initiates or stops irrigation based on the threshold set by the user.
- By utilizing the L298N motor driver and PWM signals, adjusting the pump's flow rate was made possible, allowing the user to control the water output either manually or automatically.
- With the help of the water level sensor and current sensor, the system is protected against abnormal conditions such as low water levels in the tank or unauthorized pump current surges, and alerts the user accordingly.
- The integration of the ESP8266 WiFi module and the RemoteXY application added remote control and monitoring capabilities to the system, which is a significant advantage over simple irrigation systems.

